

PROJE SPESİFİKASYONU · V1.0

Schema Merger

Farklı Excel/CSV dosyalarındaki heterojen tabloları, kullanıcı onaylı bir eşleştirme planı üzerinden tek bir dikey birleşik dosyaya dönüştüren AI-destekli veri entegrasyon aracı.

Yaklaşım: Çekirdek + CLI (önce), Web UI (sonra) · **Stack:** Python · pandas/polars · LLM (öneri katmanı)
Dağıtım: Açık kaynak (GitHub, MIT) — self-hosted, kullanıcının kendi API key'i

İçindekiler

1. Problem ve Amaç

Çözülen sorun, hedef kitle, temel değer önerisi

2. Kapsam Kararları

MVP'de olan / olmayan — kesinleşmiş sınırlar

3. Mimari Genel Bakış

İki fazlı pipeline, LLM'in rolü, bileşenler

4. Çekirdek Kavramlar

Dikey birleştirme, confidence/status, provenance, HITL

5. İki Fazlı Akış

Faz 1 analiz + onay, Faz 2 deterministik uygulama

6. Veri Sözleşmeleri

schema.yaml ve mapping.yaml formatları

7. Bileşen Detayları

Profiler, Matcher, Entity Resolution, Validator, Transformer, Writer

8. Proje Yapısı

Dizin ağacı ve dosya sorumlulukları

9. Güvenlik: API Key Yönetimi

Key'in asla repoya girmemesi — kritik kurallar

10. Geliştirme Yol Haritası

6 aşamalı, çekirdek-önce sıralama

11. Claude Code İçin Talimatlar

Aşama promptları üretme + doğrulama protokolü

1. Problem ve Amaç

Şirketlerde ve kurumsal ortamlarda veri çoğunlukla Excel veya SQL formatında saklanır. Ancak aynı türden veri, farklı dosyalarda tutarsız biçimlerde bulunur:

- Farklı sütun isimleri (fiyat / unit_price / birim_fiyat / UF)
- Ekstra veya eksik sütunlar
- Aynı ürünün farklı yazımları (Coca Cola 330ml / Coca-Cola Kutu 33cl)
- Farklı tip ve format (Türkçe "12,50" string vs. 12.50 float)

Bu tutarsızlıklar yüzünden farklı dosyaları tek bir dosyada birleştirmek elle, hataya açık ve yavaş bir iştir.

Amaç

Kullanıcı, girdi olarak dosyalarını ve çıktıda istediği hedef şemayı (sütun isimleri) verir. Sistem, doğru kaynak sütunları hedefe eşleştirir, riskli/belirsiz eşleştirmeleri kullanıcı onayına sunar, ve onaylanan plana göre verileri **bozmadan** tek bir dosyada (Excel/CSV/SQL) dikey olarak birleştirir.

EN KRİTİK KISIT

Verinin bozulmaması ve yanlış sütunların sessizce birleştirme kısmına girmemesi projenin kalbidir. LLM iki sütunu yanlış eşleştirip kendinden emin şekilde birleştirirse veri *sessizce* bozulur. Bu yüzden her eşleştirme confidence + gerekçe taşır ve riskliler insan onayına sunulur (human-in-the-loop).

Hedef Kitle ve Dağıtım

Herkese açık, self-hosted açık kaynak araç (GitHub, MIT lisansı). SaaS değil — kullanıcı kendi makinesinde çalıştırır, verisi kendinde kalır ("veriniz bize gitmiyor" — kurumsal veride büyük artı). LLM çağrıları için kullanıcı kendi API key'ini girer.

2. Kapsam Kararları (Kesinleşmiş)

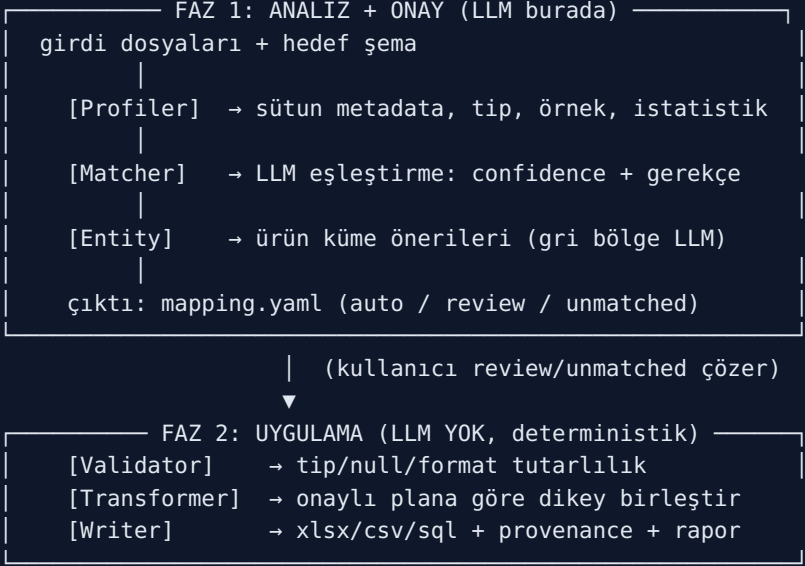
Konu	MVP'de VAR	Kapsam DIŞI (v2)
Birleştirme türü	Sadece dikey (union / append) — farklı dosyalardaki satırları alt alta tek listede toplama	Yatay join (anahtar sütun üzerinden yan yana getirme)
Girdi formatı	.xlsx , .csv	Canlı SQL veritabanı bağlantısı, .sql dump
Çıktı formatı	.xlsx , .csv , basit SQL script (CREATE TABLE + INSERT)	Canlı DB'ye yazma
Sütun eşleştirme	LLM destekli, confidence + gerekçe, dosya başına sabit çağrı	—
Entity resolution (ürün tekilleştirme)	Katmanlı: normalize → blocking → embedding → gri bölge LLM; küme bazlı onay	—
Arayüz	Önce çekirdek + CLI	Web UI (çekirdek stabilleşince)
Çakışan değerler	Hepsini tut + provenance ile işaretle	Otomatik "doğrusunu seçme"

ÖLÇEK PRENSİBİ

"Ölçek kısıtlamamalı" kararı, LLM'i satır seviyesinden çıkarmayı zorunlu kılar. LLM yalnızca **sütun eşleştirmede** (satır sayısından bağımsız) ve entity resolution'ın **gri bölgesinde** (toplam kayıtların ~%1–5'i) kullanılır. Asıl satır işleme deterministik koddur. Böylece 1.000 satır da 10.000.000 satır da aynı LLM maliyetini taşır.

3. Mimari Genel Bakış

Sistem bir **graph gerektirmez** — doğrusal, iki fazlı bir pipeline'dır. LangGraph gibi bir framework burada overkill olur; düz Python fonksiyonları yeterlidir. (Doğru aracı seçmek — gereksiz karmaşıklıktan kaçınmak — projenin olgunluk sinyalidir.)



NEDEN BU TASARIM SAĞLAM

Şeffaf: hiçbir şey arkadan sessizce yapılmaz, her karar yazılı planda. **Tekrarlanabilir:** aynı mapping.yaml → aynı sonuç, otomasyona uygun. **Test edilebilir:** Faz 2 saf deterministik kod, unit test yazılır. **Ucuz:** LLM maliyeti satır sayısından bağımsız.

4. Çekirdek Kavramlar

Dikey Birleştirme (Union)

Dosya A'da 1.000 ürün, Dosya B'de 500 ürün → hedef şemaya eşleştirilip alt alta yığılır → 1.500 satır. Entity resolution burada "aynı ürün iki dosyada da varsa tekilleştir/normalize et" görevini üstlenir.

Eşleştirme Sinyalleri (sadece isim değil)

Sütun eşleştirme yalnızca isme dayanmaz; isim yanıltıcıdır. Dört katman birlikte değerlendirilir:

1. Sütun ismi (embedding benzerliği)
2. Veri tipi (int, date, string, decimal)
3. Örnek değerler (ilk N satır — LLM'e mutlaka verilir)
4. İstatistiksel profil (unique count, null oranı, min/max, format pattern)

Üç Durum (Status)

Status	Anlamı	Davranış
auto	Yüksek güven — isim + tip + örnek değerler uyuşuyor	Otomatik eşleşir, kullanıcı dokunmasa da olur
review	Düşük/orta güven — belirsiz, riskli	Kullanıcı onaylamalı veya düzeltmeli; çözülmeden apply durur
unmatched	Bu dosyada karşılık yok	Kullanıcı elle sütun seçer ya da "boş bırak" der

Human-in-the-Loop (HITL)

Sistem tam otomatik **değildir** — bu bir zayıflık değil, satış noktasıdır. Kurumsal veride kimse kör güvenmez. Riskli eşleştirmeler kullanıcı onayına sunulur. CLI'da bu onay `mapping.yaml` dosyası üzerinden yapılır (interaktif soru-cevap yerine — ölçeklenir, versiyonlanır, otomasyona uyar).

Provenance (Köken İzleme)

Birleşik dosyadaki her satıra hangi kaynak dosyadan ve hangi orijinal sütun isminden geldiği bilgisi eklenir. Veri bozulursa geri izlenebilir — kurumsal kullanımda neredeyse zorunlu.

DEĞİŞMEZ KURAL: LLM VERİYİ ELLE İŞLEMEZ

LLM asla gerçek veri satırlarını yazmaz/dönüştürmez. LLM yalnızca **mapping planını** (hangi sütun → hangi sütun, hangi dönüşüm) üretir. Asıl satır işlemeyi deterministik kod yapar. Bu ayırım, verinin bozulmamasının tek garantisidir ve halüsinasyon riskini sıfırlar.

5. İki Fazlı Akış (Detay)

Faz 1 — Analiz ve Öneri

Sistem çalışır, **hiçbir şeyi birleştirmez**, sadece bir plan üretir. Konsola özet basar:

```
✓ 8 sütun otomatik eşleştirdi
△ 3 sütun onay bekliyor (review)
x 1 sütun hiçbir dosyada bulunamadı
→ Planı düzenle: mapping.yaml, sonra: merger apply
```

Faz 2 — Uygulama

Kullanıcı planı düzenledikten sonra **apply** çalıştırır. Sistem artık **LLM kullanmaz**. İki koruma zorunludur:

- Hâlâ **review** durumunda satır varsa **apply durur** ve uyarır — kör birleştirme yok.
- Provenance sütunları eklenir.

Çıktı iki dosyadır: **merged.xlsx** (temiz birleşik veri) ve **merge_report.xlsx** (neyin nasıl eşleştiği, kaç satır geldi, kaç null oluştu, entity-resolution'da belirsiz kalan ürünler).

Kullanıcı Yeni Sütun İsmi İsterse

İki durum ayrılır:

1. **Mevcut bir sütun seçiliyor** (dropdown/YAML): LLM'e gerek yok, kod direkt eşler. Bedava, anında.
2. **Serbest metin yeni isim yazılıyor** ("bana **vergi_dahil_fiyat** lazım"): katmanlı arama — önce tam/normalize eşleşme (kod), sonra fuzzy + embedding (ucuz), en son LLM (gri bölge). LLM burada da **karar verici değil, öneri getirici**; sonuç yine kullanıcı onayına gider.

6. Veri Sözleşmeleri

Hedef Şema — `schema.yaml`

```
target_columns:
  - name: product_name
    type: string
    required: true
  - name: unit_price
    type: decimal
    required: true
  - name: stock_quantity
    type: integer
    required: false

output:
  format: xlsx          # xlsx | csv | sql
  add_provenance: true
```

Eşleştirme Planı — `mapping.yaml` (Faz 1 çıktısı, kullanıcı düzenler)

```
- target_column: unit_price
  sources:
    - file: sales_2023.xlsx
      column: "birim fiyat"
      confidence: 0.97
      status: auto
      reason: "İsim + tip (decimal) + örnek değerler uyuyor"
    - file: export_q4.csv
      column: "UF"
      confidence: 0.54
      status: review          # RİSKLİ – kullanıcı bakmalı
      reason: "İsim belirsiz; decimal ama 'unit_freight' de olabilir"
      samples: [12.50, 8.90, 15.00]

- target_column: product_name
  sources:
    - file: sales_2023.xlsx
      column: "urun"
      confidence: 0.91
      status: auto
    - file: export_q4.csv
      column: null
      confidence: 0.0
      status: unmatched       # bu dosyada karşılığı YOK
      reason: "Hedefe uyan sütun bulunamadı"
```


7. Bileşen Detayları

Bileşenlerin çoğu **deterministik koddur**. LLM sadece Matcher'da ve Entity'nin gri bölgesinde gerçekten gereklidir. "Her şey agent olsun" tuzağına düşülmez.

Profiler LLM YOK · pandas/polars

Her girdi dosyasını tarar, LLM'e girmeden önce hazırlık yapar: sütun metadeta, veri tipi, ilk N örnek değer, istatistik (unique count, null oranı, min/max, format pattern). Çok-sheet'li Excel'de sheet seçimini yönetir.

Matcher LLM burada

Hedef şema sütunlarını kaynak sütunlarla eşleştirir. Her eşleştirme için confidence + gerekçe üretir. LLM'e sütun ismiyle birlikte tip ve 5–10 örnek değer verilir. Belirsizleri review olarak işaretler.

Entity Resolution Katmanlı · LLM sadece gri bölge

Satır seviyesinde ürün tekilleştirme. LLM satır bazında **kullanılamaz** (ölçek patlar). Dört katman:

1. **Normalizasyon** (kod): lowercase, birim çevir (33cl→330ml), noktalama temizle, kısaltma aç.
2. **Blocking** (kod): tüm-çiftler yerine sadece aynı blok (ilk 3 harf / kategori / marka). Milyon×milyon → milyon×onlarca.
3. **Embedding + eşik** (ucuz): blok içi vektör benzerliği. Yüksek → aynı, düşük → farklı.
4. **LLM** (pahalı, nadir): sadece eşige yakın gri bölge çiftleri (~%1–5).

Onay **küme bazlı** yapılı: "şu 12 yazım muhtemelen aynı ürün" grubu kullanıcıya sunulur, kullanıcı onaylar/böler. Yüksek güvenli kümeler otomatik birleşir.

Validator LLM YOK

Önerilen eşleştirmeyi tip/format tutarlılığı, aykırı değer, null patlaması açısından denetler. Matcher'ın kararını denetleyen ayrı bir göz — kritik.

Transformer LLM YOK · deterministik

Onaylı plana göre dönüşümleri uygular: tip çevirme, format normalize (özellikle TR/EN sayı ve tarih formatı), dikey birleştirme. Büyük dosyalarda streaming/chunked işleme (Polars önerilir).

Writer LLM YOK

xlsx/csv/sql çıktısı + provenance sütunları + merge_report. SQL çıktısı MVP'de CREATE TABLE + INSERT script olarak basit tutulur.

LLM Katmanı Soyutlama

Key yönetimi ve model soyutlaması. OpenAI / Anthropic / Ollama (local) desteği — config'den seçilebilir. Local model (Qwen/Llama) desteği, verisini hiçbir yere göndermek istemeyen kurumlar için değerli.

8. Proje Yapısı

```
schema-merger/
├── .env.example          # boş key şablonu (repoda VAR)
├── .gitignore            # .env İLK SATIR (key repoya girmez)
├── LICENSE               # MIT
├── README.md             # ne yapar + kurulum + key uyarısı
├── pyproject.toml
├── core/
│   ├── profiler.py      # dosya → metadata, tip, örnek, istatistik
│   ├── matcher.py       # hedef ↔ kaynak eşleştirme (LLM)
│   ├── entity.py        # normalize + blocking + embedding + gri LLM
│   ├── validator.py     # tip/null/format tutarlılık
│   ├── transformer.py   # deterministik dikey birleştirme
│   ├── writer.py        # xlsx/csv/sql + provenance + rapor
│   └── llm.py           # key yönetimi, model soyutlama
├── cli/
│   └── main.py          # analyze / apply komutları
└── tests/
```

MİMARİ İLKE

UI ve CLI **aynı çekirdeği** kullanır. Çekirdek = Python kütüphanesi (dosya + hedef şema alır → plan JSON verir; plan + onay alır → birleşik dosya verir). CLI ve UI sadece "sunum" katmanıdır, iş mantığı iki kez yazılmaz.

İki Komutluk CLI

```
# Faz 1: analiz – plan üretir, birleştirmez
merger analyze --inputs sales_2023.xlsx export_q4.csv \
               --target-schema schema.yaml \
               --out mapping.yaml

# (kullanıcı mapping.yaml'daki review/unmatched satırlarını düzenler)

# Faz 2: uygula – LLM yok, deterministik
merger apply --mapping mapping.yaml \
             --out merged.xlsx --format xlsx
```

9. Güvenlik: API Key Yönetimi

MUTLAK KURAL

Hiçbir API key ASLA repoya girmez, ASLA commit'lenmez. GitHub'a proje koyan çok kişinin yandığı yer burasıdır.

- Key `.env` dosyasından okunur (`python-dotenv`).
- `.env` ilk iş `.gitignore` 'a eklenir — repo başlamadan önce. Bir kez commit'lenirse git geçmişinden temizlemek baş belasıdır.
- Repoda `.env.example` bulunur (boş anahtarlar) — hem güvenli hem kurulum dokümanı.
- Kod key'i ortam değişkeninden alır, asla hardcode etmez. Key yoksa açık hata: "OPENAI_API_KEY tanımlı değil, `.env` dosyasını kontrol et."
- Bonus: commit öncesi `git-secrets` / `pre-commit` hook ile sızıntı taraması.
- README'de net not: "Kendi key'inizi girin, bize gitmez."

MVP'de Basit Tutulacak Kenar Durumlar

- **Çok-sheet'li Excel:** kullanıcı sheet belirtir ya da hepsi taranır (kararlaştırılıp dokümante edilir).
- **Çakışan değerler:** hepsi tutulur + provenance ile işaretlenir; otomatik seçim yok.
- **Tip çatışmaları:** TR/EN sayı-tarih normalizasyonu Transformer'da olmazsa olmaz (kullanıcı kitlesi Türkçe veri ile çalışır).

10. Geliştirme Yol Haritası (Çekirdek-Önce)

1 İskelet + Veri Katmanı

Proje yapısı, `.gitignore` / `.env` güvenliği, Profiler (dosya okuma + profilleme). LLM yok, saf pandas/polars. TR/EN tip normalizasyonu baştan konur.

Çıktı: çalışan profiler, güvenli repo iskeleti

2 Matcher + Plan Üretimi

LLM ile sütun eşleştirme, `mapping.yaml` çıktısı, confidence/status/gerekçe. `analyze` komutu çalışır.

Çıktı: analyze komutu + mapping.yaml üretimi

3 Transformer + Writer

Onaylı planı deterministik uygula, dikey birleştir, provenance ekle, xlsx/csv/sql + rapor yaz. **apply** komutu tamam. **Bu noktada uçtan uca çalışan MVP var** (entity resolution olmadan bile kullanılabilir).

Çıktı: uçtan uca çalışan MVP

4 Entity Resolution

Normalize + blocking + embedding + gri bölge LLM, küme raporu. Ürün tekilleştirme devreye girer. (En zor ve en riskli parça — çekirdek onsuz oturduktan sonra eklenir.)

Çıktı: ürün tekilleştirme + küme onayı

5 Validator + Cilalama

Tutarlılık kontrolleri, kenar durumlar, testler, README, LICENSE.

Çıktı: sağlamlaştırılmış, dokümante edilmiş sürüm

6 Web UI

FastAPI + React, çekirdeği saran onay ekranı: yeşil/sarı/kırmızı kartlar, dropdown ile düzeltme, "Birleştir" butonu ancak review kalmadığında aktif.

Çıktı: teknik olmayan kullanıcı için görsel arayüz

11. Claude Code İçin Talimatlar

Bu belge, projenin tek referans kaynağıdır. Claude Code'dan beklenen çalışma şekli:

A. Aşama Promptu Üretme

Her yol haritası aşaması için, ayrı bir kod ajanına (Codex) verilecek **kendi kendine yeten bir prompt** hazırla. Her prompt şunları içermeli:

- Aşamanın amacı ve bu belgedeki ilgili kararlara referans (kapsam, veri sözleşmeleri, değişmez kurallar).
- Tam olarak hangi dosyaların oluşturulacağı/değiştirileceği.
- Kabul kriterleri (aşamanın "tamamlandı" sayılması için sağlanması gereken somut, doğrulanabilir koşullar).
- Değişmez kısıtların hatırlatması: LLM veriyi elle işlemez; key repoya girmez; sadece dikey birleştirme; LLM satır seviyesinde çalışmaz.
- O aşamada yazılması gereken testler.

B. Doğrulama Protokolü (Her Yeniden Başlangıçta)

Codex her yeni oturuma başladığında, yeni aşamaya geçmeden önce **önceki aşamanın gerçekten tamamlandığını doğrulamalı**. Bunun için her aşama promptunun sonunda bir "Önceki Aşama Doğrulama Kontrol Listesi" bulunmalı:

- Önceki aşamanın kabul kriterleri tek tek kontrol edilir (dosyalar var mı, testler geçiyor mu, komut çalışıyor mu).
- Eksik varsa önce o giderilir, sonra yeni aşamaya geçilir.
- Doğrulama sonucu kısa bir raporla özetlenir (neyin tamam, neyin eksik olduğu).

CLAUDE CODE'DAN İLK İSTEK

Bu belgeyi oku, projeyi tam kavradığını kısa bir özetle doğrula, sonra **Aşama 1** için Codex'e verilecek ilk promptu üret — kabul kriterleri ve (Aşama 1 olduğu için boş geçilebilecek ama formatı gösteren) önceki-aşama doğrulama bölümü dahil.

Değişmez Kararlar Özeti (hiçbir aşamada ihlal edilmez)

#	Karar
1	Sadece dikey (union) birleştirme.
2	İki fazlı akış: analyze (LLM) → kullanıcı onayı → apply (LLM yok).
3	LLM yalnızca sütun eşleştirmede ve entity gri bölgesinde; asla satır verisini elle işlemez.
4	Her eşleştirme confidence + status (auto/review/unmatched) + gerekçe taşır.
5	apply, review kalırsa durur — kör birleştirme yok.
6	Provenance sütunları her zaman eklenir.
7	API key asla repoya girmez (.env + .gitignore + .env.example).
8	Çekirdek + CLI önce, UI sonra; UI ve CLI aynı çekirdeği paylaşır.